

Name:K.KEERTHAN

Hallticket:2303A52008

Assginment:15.1

Task1:

ask 1: Initial API Setup for a Service Platform

Scenario

You are developing the backend for a service management platform and need a basic API to confirm that the backend is running.

Task Description

- Use AI assistance to generate a basic Flask application.
- Create a root endpoint /status that returns a JSON message indicating the API is live.
- Run the server locally and verify the response using a browser or API testing tool.

Expected Output

- Accessing /status returns:

```
{"status": "API is running"}
```
- Flask server runs without errors.

Prompt:create Api using node js

Code

```
JS server1.js > ...
1  const express = require('express');
2
3  const app = express();
4  const PORT = 3000;
5
6  app.get('/', (req, res) => {
7    res.send("Welcome to the API");
8  });
9
10 app.get('/status', (req, res) => {
11   res.json({ status: "API is running" });
12 });
13
14 app.listen(PORT, () => {
15   console.log(`Server running at http://localhost:${PORT}`);
16 });
```

```
Node.js v24.13.0
(.venv) PS C:\Users\gadda\OneDrive\Desktop\Ai Assitant coding> node server1.js
Server running at http://localhost:3000
(.venv) PS C:\Users\gadda\OneDrive\Desktop\Ai Assitant coding> node server1.js
Server running at http://localhost:3000
```

Generate Commit Message

Start JSON Server

Ln 16, Col 4

Spaces: 2

Output:



Observation:

The Flask API was successfully set up and ran locally without any errors.

The /status endpoint responded correctly with the expected JSON message: {"status": "API is running"}.

The API was verified using a browser/API tool, confirming the backend is live and functioning properly.

Task:02

Create API to Add and View Resources

Scenario

Your backend needs to manage customer service requests stored temporarily in memory.

Task Description

- Use AI to help implement:

- o A GET endpoint to retrieve all service requests
- o A POST endpoint to add a new request
- Each request should contain title and priority.

Expected Output

- GET /requests returns an empty list initially
- POST /requests with:

```
{"title": "Internet issue", "priority": "High"}
```

returns confirmation with the added object

- Subsequent GET returns the added request

Prompt :create API customer service required

Code:

```
js server1.js > ...
1  const express = require('express');
2
3  const app = express();
4  const PORT = 3000;
5
6  // Middleware to parse JSON
7  app.use(express.json());
8
9  // In-memory storage
10 let requests = [];
11
12 // Root route
13 app.get('/', (req, res) => {
14   res.send("Welcome to the Service API");
15 });
16
17 // GET all requests
18 app.get('/requests', (req, res) => {
19   res.json(requests);
20 });
21
22 // POST new request
23 app.post('/requests', (req, res) => {
24   const { title, priority } = req.body;
25
26   requests.push({ title, priority });
27   res.json(requests);
28 });
29
```

OUTPUT DEBUG CONSOLE TERMINAL 7 PORTS SPELL CHECKER 2

TERMINAL

```
(.venv) PS C:\Users\gadda\OneDrive\Desktop\Ai Assitant coding> node server.js
at node:internal/main/run_main_module:33:47 {
  code: 'MODULE_NOT_FOUND',
  requireStack: []
}

Node.js v24.13.0
(.venv) PS C:\Users\gadda\OneDrive\Desktop\Ai Assitant coding> node server1.js
Server running at http://localhost:3000
```

SESSIONS

No matching sess

You've reached the

+ js server1.js

Describe what to

+ Auto

Local

Generate Commit Message Start JSON Server Ln 50, Col 4 Spaces: 2 UTF-8 CRLF {} JavaScript Chat quota reached

Output:

Pretty-print ☐

```
[{"id":1,"title":"Internet issue","priority":"High"}]
```

Observation:

The API successfully supports adding new service requests and retrieving them using POST and GET endpoints.

Initially, the GET /requests endpoint returns an empty list, confirming proper initialization of in-memory storage.

After adding a request via POST, the data is stored correctly and reflected in subsequent GET responsesTask:03

Task 3: Modify Existing Resource Using AI Assistance

Scenario

Service requests may change in priority as issues are escalated.

Task Description

- Use AI to generate a PUT endpoint that updates a request based on its index.
- Handle invalid indexes gracefully with proper error responses.
- Ensure the updated request is returned in the response.

Expected Output

- Valid update returns:

```
{"message": "Request updated", "request": {...}}
```

- Invalid index returns a clear error message with HTTP 404.

Prompt:create modify the using ai assitence

Code:

```
// ✅ PUT - Update request by ID
app.put('/requests/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const { title, priority } = req.body;

  // Find request
  const request = requests.find(r => r.id === id);

  // ❌ If not found
  if (!request) {
    return res.status(404).json({
      message: "Request not found"
    });
  }

  // ✅ Update values (only if provided)
  if (title) request.title = title;
  if (priority) request.priority = priority;

  res.json({
    message: "Request updated",
    request: request
  });
});
```

DEBUG CONSOLE **TERMINAL 7** PORTS SPELL CHECKER 2

Output:

```
Pretty-print ☒
[
  {
    "id": 1,
    "title": "Internet issue",
    "priority": "High"
  }
]
```

Observation:

- The PUT endpoint successfully updates service requests based on the given index and returns the modified data.
- Proper error handling is implemented, returning a clear 404 response when an invalid index is provided.

- The API ensures data consistency by updating in-memory storage and confirming changes in the response.

Task:04

ask 4: Remove a Resource Safely

Scenario

Resolved service requests must be removed from the active list.

Task Description

- Use AI assistance to implement a DELETE endpoint.
- The endpoint should remove a request by index.
- Return the deleted request details in the response.

Expected Output

- Successful deletion returns confirmation and removed data
- Invalid index returns an error message without crashing the server

Prompt:

Resolved service requests must be removed from the active list.

Code:

```
// ✅ DELETE - Remove request by INDEX
app.delete('/requests/:index', (req, res) => {
  const index = parseInt(req.params.index);

  // ❌ Check invalid index
  if (isNaN(index) || index < 0 || index >= requests.length) {
    return res.status(404).json({
      message: "Invalid index. Request not found"
    });
  }

  // ✅ Remove request
  const deletedRequest = requests.splice(index, 1)[0];

  res.json({
    message: "Request deleted successfully",
    request: deletedRequest
  });
});
```

Output:



Observation:

- The DELETE endpoint successfully removes a service request by index and returns the deleted request details.
- Proper error handling ensures that invalid indexes return a clear error message without crashing the server.
- The API maintains stability and consistency by safely updating the in-memory data after deletion.

Task:05

ask 5: AI-Assisted API Documentation and Readability

Scenario

Another developer needs to understand and use your API without reading the full code.

Task Description

- Use AI to:
 - o Add meaningful docstrings and inline comments to all endpoints
 - o Clearly describe request methods, inputs, and responses
- Optionally, use AI guidance to integrate automatic API documentation (Swagger or Flask-RESTX).

Expected Output

- Well-documented endpoints with readable docstrings
- If implemented, a documentation UI displaying:
 - o Endpoint paths
 - o HTTP methods
 - o Sample request/response formats

Prompt:

Code:

```
server2.js > ...
1  /**
2   * Service Management API
3   * -----
4   * This API allows users to:
5   * - View service requests
6   * - Add new requests
7   * - Update existing requests
8   * - Delete resolved requests
9   *
10  * Base URL: http://localhost:3000
11  */
12
13  const express = require('express');
14  const app = express();
15  const PORT = 3000;
16
17  /**
18   * Middleware
19   * Parses incoming JSON requests
20   */
21  app.use(express.json());
22
23  /**
24   * In-memory storage for service requests
25   * Each request contains:
```

Output:



Observation :

- All API endpoints are enhanced with clear docstrings and inline comments, making them easy to understand without reading the full code.
- Each endpoint now clearly specifies HTTP methods, input formats, and expected responses for better usability.
- Optional Swagger/Flask-RESTX integration provides a user-friendly documentation UI with endpoint paths, methods, and sample request/response details.